

TMC-305

Full Stack Web Development

M.C.A. (Third Semester) - Mid Semester Examination, 2022

Time: 1½ Hours | Maximum Marks: 50

Complete Solved Question Paper
with detailed answers, programs and revision notes

Instructions in the paper: (i) Answer all the questions by choosing any *one* of the sub-questions. (ii) Each question carries 10 marks. In this document **both** options (a) and (b) of every question are solved so that the full syllabus can be revised.

Q. No.	Topic	Marks	CO
1 (a) / (b)	HTML5 Drag and Drop MathML	10	CO1
2 (a) / (b)	SVG and cubic Bezier curve SVG vs Canvas, linear gradient	10	CO1
3 (a) / (b)	Bootstrap CSS CSS 2D and 3D transforms	10	CO2
4 (a) / (b)	Media queries (paged media) CSS3 gradients	10	CO2 / CO1
5 (a) / (b)	HTML5 audio and video Responsive page and Bootstrap gallery	10	CO2
Bonus	Quick revision notes on the full syllabus	-	-

Note: Image files used in the programs (image1.jpg, song.mp3, movie.mp4, etc.) are placeholders - keep your own files in the same folder as the HTML file. Bootstrap examples need an internet connection for the CDN links.

Q.1 (a)

Question: What is the use of drag and drop features of HTML5? Create two images and drag the drop the images on the same web page.

10 Marks | CO1

Answer:

1. Introduction

Drag and Drop (DnD) is a native HTML5 feature that lets a user **grab** an element with the mouse (or finger on touch devices), **move** it across the page and **release** it over another element. Before HTML5, developers had to depend on JavaScript libraries such as jQuery UI or on Flash. HTML5 provides a built-in **Drag and Drop API**, which is part of the HTML Living Standard and is supported by all modern browsers (Chrome, Firefox, Edge, Safari).

2. Uses of the Drag and Drop feature

- **File upload:** the user drags files from the desktop and drops them on a page area (as in Gmail, Google Drive) instead of using the *Browse* button.
- **Reordering lists:** re-arranging to-do items, playlists, menu items, or Kanban cards (Trello style).
- **Shopping carts:** dragging a product image into the cart icon.
- **Games and puzzles:** jigsaw puzzles, card games, chess-like pieces moved between boxes.
- **Dashboards / page builders:** dragging widgets, charts, or form fields to design a layout.
- **Data transfer between elements or windows:** text, links, images and files can be dragged between two browser windows or from another application.
- It gives a **better user experience** and reduces code because no third-party plug-in is required.

3. How it works - the draggable attribute

Any element can be made draggable by setting the global attribute `draggable="true"`. Images, links and selected text are draggable by default. The element that is dragged is the **source** and the element on which it is released is the **drop target**.

4. Drag and Drop events

Event	Fired on	Purpose
dragstart	Dragged element	Fires when the user starts dragging. Used to store the data with <i>setData()</i> .
drag	Dragged element	Fires repeatedly while the element is being dragged.
dragenter	Drop target	Fires when the dragged element enters a valid target.
dragover	Drop target	Fires continuously while it is over the target. Default action must be cancelled with <i>preventDefault()</i> to allow a drop.
dragleave	Drop target	Fires when the dragged element leaves the target.
drop	Drop target	Fires when the element is released on a valid target. Data is read here.
dragend	Dragged element	Fires when dragging finishes (successfully or cancelled).

5. The dataTransfer object

The event .dataTransfer object carries the data being dragged from source to target.

Member	Description
setData(type, data)	Stores data (e.g. id of the element) in the given format, such as "text" or "text/plain".
getData(type)	Retrieves the stored data inside the drop event.
effectAllowed	Which operation is allowed: copy, move, link, all, none.
dropEffect	The operation chosen on the target: copy, move, link or none.
setDragImage(img, x, y)	Sets a custom image that follows the cursor during dragging.
files	List of files being dragged from the operating system (used in file upload).

6. Steps to implement

- Make the element draggable: `draggable="true"`.
- Handle *ondragstart*: store the id of the dragged element using `setData()`.
- Handle *ondragover* on the drop zone: call `event.preventDefault()` (by default, elements do not accept drops).
- Handle *ondrop* on the drop zone: call `preventDefault()`, read the id with `getData()` and append that element to the target using `appendChild()`.

7. Program: two images dragged and dropped between two boxes

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>HTML5 Drag and Drop</title>
<style>
    .box { width: 230px; height: 190px; border: 2px dashed #555; margin: 10px;
        display: inline-block; vertical-align: top; text-align: center;
        padding: 5px; background: #fafafa; }
    .box img { width: 100px; height: 100px; margin: 5px; cursor: move; }
</style>
</head>
<body>
    <h2>Drag the two images and drop them in any box</h2>

    <div id="box1" class="box" ondrop="drop(event)" ondragover="allowDrop(event)">
        
        
    </div>

    <div id="box2" class="box" ondrop="drop(event)" ondragover="allowDrop(event)">
        <p>Drop here</p>
    </div>

<script>
    function allowDrop(ev) {          // dragover : allow dropping
        ev.preventDefault();
    }
    function drag(ev) {                // dragstart: remember which element
        ev.dataTransfer.setData("text", ev.target.id);
    }
    function drop(ev) {                // drop      : move element into the box
        ev.preventDefault();
        var id = ev.dataTransfer.getData("text");
        var target = ev.target;
        while (!target.classList.contains("box")) {    // if dropped on an image,
            target = target.parentNode;                // go up to the box
        }
        target.appendChild(document.getElementById(id));
    }
</script>
</body>
</html>

```

8. Explanation of the program

- Two *img* elements (image1.jpg and image2.jpg) are placed in **box1**; both have *draggable="true"* so the browser lets them be picked up.
- **drag(ev)** runs on *dragstart*; it saves the id of the image in the *dataTransfer* object.
- **allowDrop(ev)** runs on *dragover*; without *preventDefault()* the box would refuse the drop.
- **drop(ev)** runs on *drop*; it reads the saved id, finds the element and appends it into the target box - so the image visually moves. Both images can be dragged back and forth between box1 and box2 on the same page.

9. Advantages and limitations

Advantages	Limitations
Native support, no plug-ins or libraries needed.	Old touch devices/mobile browsers have weak support (touch events may be needed).
Simple JavaScript API with clear events.	Default drag image cannot be styled fully without <i>setDragImage()</i> .
Works with text, images, links and OS files.	Accessibility (keyboard use) must be added manually.

Q.1 (b)

Question: What is MathML in HTML5? Print the following equations using HTML5 code: (i) $(a + b)^2 = a^2 + b^2 + 2ab$ (ii) $a/b - c/d = e/f$

10 Marks | CO1

Answer:

1. What is MathML?

MathML (Mathematical Markup Language) is an XML-based markup language, recommended by the W3C, for describing mathematical notation and capturing both its **structure** and **content**. HTML5 allows MathML to be embedded directly in a web page with the `<math>` element - no image or plug-in is required. All MathML tags live inside the *math* root element, and the equation remains searchable, scalable and accessible to screen readers (unlike an image of an equation).

2. Features / advantages of MathML

- Native browser rendering (Firefox, Safari, and Chrome from version 109 support MathML Core).
- Text is real content - it can be copied, searched, resized, and read by screen readers.
- Can be styled with CSS and manipulated with JavaScript/DOM.
- Better quality than equation images at every zoom level; smaller file size.

3. Important MathML elements

Element	Meaning	Example
<code><math></code>	Root element of every equation. Attribute <code>display="block"</code> centres it.	<code><math display="block"></code>
<code><mi></code>	Identifier - variables, function names.	<code><mi>a</mi></code>
<code><mn></code>	Number.	<code><mn>2</mn></code>
<code><mo></code>	Operator, bracket or fence.	<code><mo>+</mo></code>
<code><mrow></code>	Groups sub-expressions horizontally.	<code><mrow>...</mrow></code>
<code><msup></code>	Superscript (base, exponent).	<code><msup><mi>a</mi><mn>2</mn></msup></code>
<code><msub></code>	Subscript (base, subscript).	x_1
<code><mfrac></code>	Fraction (numerator, denominator).	<code><mfrac><mi>a</mi><mi>b</mi></mfrac></code>
<code><msqrt></code>	Square root.	<code><msqrt><mi>x</mi></msqrt></code>
<code><mroot></code>	n-th root.	cube root, etc.
<code><mtext></code>	Plain text inside a formula.	"where x is real"
<code><mtable></code>	Table / matrix (with <code>mtr</code> rows and <code>mttd</code> cells).	matrices

4. Equation (i): $(a + b)^2 = a^2 + b^2 + 2ab$

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>MathML</title></head>
<body>
<h3>Equation (i)</h3>
<math xmlns="http://www.w3.org/1998/Math/MathML" display="block">
  <msup>
    <mrow>
      <mo>(</mo><mi>a</mi><mo>+</mo><mi>b</mi><mo>)</mo>
    </mrow>
    <mn>2</mn>
  </msup>
  <mo>=</mo>
  <msup><mi>a</mi><mn>2</mn></msup>
  <mo>+</mo>
  <msup><mi>b</mi><mn>2</mn></msup>
  <mo>+</mo>
  <mn>2</mn><mi>a</mi><mi>b</mi>
</math>
```

5. Equation (ii): $a/b - c/d = e/f$

```
<h3>Equation (ii)</h3>
<math xmlns="http://www.w3.org/1998/Math/MathML" display="block">
  <mfrac><mi>a</mi><mi>b</mi></mfrac>
  <mo>&minus;</mo>
  <mfrac><mi>c</mi><mi>d</mi></mfrac>
  <mo>=</mo>
  <mfrac><mi>e</mi><mi>f</mi></mfrac>
</math>

</body>
</html>
```

6. Explanation

- **msup** takes exactly two children: the base and the exponent. For $(a + b)^2$ the base is a **mrow** that groups $(a + b)$ so that the exponent applies to the whole bracket.
- **mi** is used for the letters a, b; **mn** for the numbers; **mo** for +, =, brackets and the minus sign (*−* gives a proper mathematical minus).
- **2ab** is written as *mn* followed by two *mi* elements - invisible multiplication.
- **mfrac** takes numerator and denominator, so a/b , c/d and e/f are displayed as stacked fractions.

7. Expected output

Equation	Rendered result in browser
(i)	$(a + b)^2 = a^2 + b^2 + 2ab$
(ii)	fraction a over b, minus fraction c over d, equals fraction e over f

Exam tip: Also mention that MathML differs from LaTeX/images: it is part of the DOM, so it is styled with CSS and is accessible. Older browsers may need the MathJax library as a fallback.

Q.2 (a)

Question: What is the use of SVG in HTML5? Create a cubic Bezier curve on the same path using SVG. Also specify the height and the width of the SVG to be 300 units.

10 Marks | CO1

Answer:

1. What is SVG?

SVG (Scalable Vector Graphics) is an XML-based language for describing two-dimensional vector graphics. In HTML5 an SVG can be written directly inside the page using the `<svg>` element (**inline SVG**) or included through `<img>`, `<object>` or CSS background. Because shapes are stored as mathematical descriptions and not pixels, an SVG can be scaled to any size without losing quality.

2. Uses of SVG in HTML5

- **Resolution independent** logos, icons and illustrations that stay sharp on Retina/4K screens.
- **Charts, graphs and data visualisation** (D3.js, Chart libraries generate SVG).
- **Maps, diagrams, flowcharts** and floor plans.
- **Animations and interactivity:** each shape is a DOM element, so CSS, SMIL animations and JavaScript event handlers can be attached to it.
- Small file size for simple graphics; text inside SVG is searchable and selectable (good for SEO and accessibility).
- Supports **filters, gradients, patterns, clipping and masking**.

3. Basic SVG shapes

Element	Purpose	Main attributes
<code><rect></code>	Rectangle	x, y, width, height, rx, ry
<code><circle></code>	Circle	cx, cy, r
<code><ellipse></code>	Ellipse	cx, cy, rx, ry
<code><line></code>	Straight line	x1, y1, x2, y2
<code><polyline></code> / <code><polygon></code>	Open / closed shape with many points	points
<code><path></code>	Any shape (lines, curves, arcs)	d (path data)
<code><text></code>	Text	x, y, font-size

4. The path element and its commands

The `<path>` element draws shapes using the `d` attribute. Upper-case commands use absolute coordinates; lower-case commands use relative coordinates.

Command	Name	Meaning
M x y	Move to	Moves the pen to a point without drawing.
L x y	Line to	Draws a straight line to the point.
H x / V y	Horizontal / vertical line	Draws a horizontal or vertical line.

Command	Name	Meaning
C x1 y1, x2 y2, x y	Cubic Bezier curve	Two control points (x1,y1), (x2,y2) and an end point (x,y).
S x2 y2, x y	Smooth cubic Bezier	Reflects the previous control point automatically.
Q x1 y1, x y	Quadratic Bezier	One control point and an end point.
T x y	Smooth quadratic Bezier	Shorthand for a continuing quadratic curve.
A rx ry rot large sweep x y	Elliptical arc	Draws an arc of an ellipse.
Z	Close path	Draws a straight line back to the starting point.

5. What is a cubic Bezier curve?

A cubic Bezier curve is defined by **four points**: a start point P0, two control points P1 and P2 and an end point P3. The curve starts at P0 heading toward P1 and ends at P3 arriving from the direction of P2. The control points pull the curve but the curve does not normally pass through them. In SVG it is drawn with the **C** command.

6. Program: cubic Bezier curve in a 300 x 300 SVG

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>SVG Cubic Bezier</title></head>
<body>
<svg width="300" height="300" viewBox="0 0 300 300"
    xmlns="http://www.w3.org/2000/svg" style="border:1px solid black">

    <!-- Cubic Bezier curve: M start C control1, control2, end -->
    <path d="M 20 250 C 40 20, 260 20, 280 250"
        stroke="blue" stroke-width="4" fill="none"/>

    <!-- helper lines joining end points to their control points -->
    <line x1="20" y1="250" x2="40" y2="20" stroke="grey" stroke-dasharray="5,4"/>
    <line x1="280" y1="250" x2="260" y2="20" stroke="grey" stroke-dasharray="5,4"/>

    <!-- start / end points (red) and control points (green) -->
    <circle cx="20" cy="250" r="5" fill="red"/>
    <circle cx="280" cy="250" r="5" fill="red"/>
    <circle cx="40" cy="20" r="5" fill="green"/>
    <circle cx="260" cy="20" r="5" fill="green"/>

    <text x="105" y="290" font-size="12">Cubic Bezier: M 20 250 C 40 20, 260 20, 280 250</text>
</svg>
</body>
</html>
```

7. Explanation

- **width="300" height="300"** set the size of the SVG viewport to 300 x 300 units as required. **viewBox="0 0 300 300"** makes 1 user unit = 1 pixel and keeps the drawing scalable.
- **M 20 250** places the pen at the start point P0 = (20, 250).
- **C 40 20, 260 20, 280 250** draws the cubic curve with control points P1 = (40, 20), P2 = (260, 20) and end point P3 = (280, 250). The result is a smooth arch.
- **fill="none"** is important; otherwise the browser fills the area under the curve with black.
- The dashed lines and coloured circles are only visual aids showing the control points.

8. Extra: multiple curves on the same path

To continue a curve on the same path use the **S** command, which mirrors the previous control point:

```
<path d="M 10 150 C 40 10, 110 10, 150 150 S 260 290, 290 150"
    stroke="red" stroke-width="3" fill="none"/>
```

Q.2 (b)

Question: Differentiate between SVG and Canvas. Create a rectangle with a height = 100 units and width = 200 units. Also fill the four different colors inside rectangle using linear gradient.

10 Marks | CO1

Answer:

1. Introduction

Both **SVG** and **Canvas** are HTML5 technologies for drawing graphics in the browser, but they work in completely different ways. SVG is a **vector, retained-mode** graphics format (every shape stays in the DOM). Canvas is a **raster, immediate-mode** API: JavaScript draws pixels on a bitmap and the browser forgets the shapes once they are drawn.

2. Difference between SVG and Canvas

Basis	SVG	Canvas
Type of graphics	Vector-based (shapes described by XML).	Raster / pixel-based (bitmap).
Element / language	<code><svg></code> element; markup in XML.	<code><canvas></code> element; drawn only with JavaScript.
Mode	Retained mode - every shape is remembered as a DOM object.	Immediate mode - once drawn, the shape is just pixels.
Scalability	Resolution independent; never loses quality when zoomed or resized.	Resolution dependent; zooming makes the image blurry/pixelated.
Event handling	Events (click, hover) can be attached to each individual shape.	No per-shape events; whole canvas gets the event and the code must compute which object was hit.
Styling	Can be styled by CSS.	Not styled by CSS; all properties set in JavaScript.
Performance	Slows down with a very large number of objects (thousands) since each is a DOM node.	Faster for many objects and pixel manipulation; suited to games and real-time drawing.
Animation	SMIL, CSS animations or JavaScript on DOM elements.	Redraw the whole frame using <code>requestAnimationFrame()</code> .
Modification	Easy to edit an existing shape by changing its attributes.	To change a drawing, it must be erased and redrawn.
Accessibility / SEO	Text and shapes are in the DOM, so searchable and accessible.	Content is invisible to screen readers and search engines.
Best used for	Logos, icons, maps, charts, diagrams.	Games, image processing, particle effects, real-time graphs, video effects.
Saving output	Save as .svg text file.	Export to PNG/JPEG using <code>toDataURL()</code> .

3. Linear gradient

A **linear gradient** is a smooth transition between two or more colours along a straight line. In SVG it is defined inside `<defs>` using `<linearGradient>` with a number of `<stop>` elements. The direction is set with the vector (x1, y1) to (x2, y2). Each stop has an *offset* (position 0% - 100%) and a *stop-color*. The gradient is then applied using `fill="url(#id)"`.

4. Program 1 - Using SVG (200 x 100 rectangle with four colours)

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>SVG Linear Gradient</title></head>
<body>
<svg width="300" height="150" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="fourColors" x1="0%" y1="0%" x2="100%" y2="0%">
      <stop offset="0%" stop-color="red"/>
      <stop offset="33%" stop-color="yellow"/>
      <stop offset="66%" stop-color="green"/>
      <stop offset="100%" stop-color="blue"/>
    </linearGradient>
  </defs>

  <!-- rectangle: width = 200 units, height = 100 units -->
  <rect x="50" y="25" width="200" height="100"
    fill="url(#fourColors)" stroke="black" stroke-width="2"/>
</svg>
</body>
</html>
```

Explanation

- **<defs>** stores reusable objects; the gradient is not drawn until it is referenced.
- **x1=0% ... x2=100%, y1=y2=0%** makes a horizontal gradient (left to right). Use **x1=0% y1=0% x2=0% y2=100%** for vertical or different values for diagonal.
- Four **<stop>** elements give the four colours: red, yellow, green and blue.
- **<rect width="200" height="100">** creates the required rectangle; **fill="url(#fourColors)"** paints it with the gradient.

5. Program 2 - Same rectangle using Canvas

```
<canvas id="myCanvas" width="300" height="150" style="border:1px solid #000"></canvas>
<script>
  var canvas = document.getElementById("myCanvas");
  var ctx = canvas.getContext("2d");

  // gradient from x = 50 to x = 250 (left to right)
  var grd = ctx.createLinearGradient(50, 0, 250, 0);
  grd.addColorStop(0, "red");
  grd.addColorStop(0.33, "yellow");
  grd.addColorStop(0.66, "green");
  grd.addColorStop(1, "blue");

  ctx.fillStyle = grd;
  ctx.fillRect(50, 25, 200, 100);    // x, y, width = 200, height = 100
</script>
```

6. Types of gradient in SVG

Gradient	Element	Description
Linear	<code><linearGradient></code>	Colours change along a straight line (horizontal, vertical, diagonal).
Radial	<code><radialGradient></code>	Colours change outward from a centre point (cx, cy, r).

Q.3 (a)

Question: What is the use of Bootstrap CSS? What are the advantages of using bootstrap?

10 Marks | CO2

Answer:

1. What is Bootstrap?

Bootstrap is the world's most popular free and open-source **front-end framework** for building responsive, mobile-first websites. It was originally developed at Twitter (2011) by Mark Otto and Jacob Thornton and is now maintained by an open-source community. It is a collection of ready-made **CSS classes**, **JavaScript plug-ins** and **HTML components**, so a developer can build a professional looking layout by simply adding class names to HTML elements instead of writing long CSS. The current stable series is Bootstrap 5, which no longer depends on jQuery.

2. Use of Bootstrap CSS

- **Responsive layouts:** the 12-column *grid system* adapts the page to mobile, tablet and desktop.
- **Typography and reboot:** consistent fonts, headings, lists, and a normalised base style across browsers.
- **Ready-made components:** navbar, buttons, cards, alerts, badges, carousel, modal, dropdown, accordion, pagination, progress bars, list groups, tooltips, and more.
- **Forms and tables:** styled inputs, form validation, striped/hover/bordered tables.
- **Utilities:** spacing (margin/padding), colours, borders, display, flexbox and position helper classes.
- **Images and media:** *img-fluid*, *ratio* helpers for responsive images and videos.
- **JavaScript plug-ins:** interactive behaviour (modals, carousels, collapse) using data attributes without writing JavaScript.

3. How to include Bootstrap (CDN)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <!-- viewport tag is a must for responsive behaviour -->
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
</head>
<body>
  <div class="container">
    <h1 class="text-primary">Hello Bootstrap</h1>
    <button class="btn btn-success">Click me</button>
  </div>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
  </script>
</body>
</html>
```

4. Bootstrap grid system

The layout is built with a **container**, **rows** and up to **12 columns** per row. Columns that exceed 12 wrap to the next line. Breakpoint prefixes decide when the layout changes.

Breakpoint	Class prefix	Screen width	Typical device
Extra small	col-	< 576 px	Phones (portrait)
Small	col-sm-	>= 576 px	Phones (landscape)

Breakpoint	Class prefix	Screen width	Typical device
Medium	col-md-	>= 768 px	Tablets
Large	col-lg-	>= 992 px	Laptops
Extra large	col-xl-	>= 1200 px	Desktops
XXL	col-xxl-	>= 1400 px	Large desktops

```

<div class="container">
  <div class="row">
    <div class="col-12 col-md-6 col-lg-4">Box 1</div>
    <div class="col-12 col-md-6 col-lg-4">Box 2</div>
    <div class="col-12 col-md-12 col-lg-4">Box 3</div>
  </div>
</div>

```

5. Advantages of using Bootstrap

Advantage	Explanation
Faster development	Pre-built components and classes reduce coding time and effort drastically.
Responsive, mobile-first	One code base works on phones, tablets and desktops automatically.
Cross-browser consistency	Normalises styles so the site looks the same in Chrome, Firefox, Edge, Safari.
Easy to learn and use	Only basic HTML and CSS knowledge is needed; documentation is excellent.
Customisable	Sass variables and theme builders allow changing colours, fonts and components; only needed parts can be included.
Consistency	A uniform design language throughout the project and across a team.
Large community and support	Huge number of themes, templates, tutorials and plug-ins; well maintained and regularly updated.
Free and open source	MIT licensed, usable in commercial projects.
Integration	Works with React, Angular, Vue and back-end frameworks (Node, Django, PHP, etc.).
Accessibility	Components follow ARIA guidelines to a large extent.

6. Disadvantages (for a balanced answer)

- Websites can look alike (generic) unless customised.
- Unused CSS/JS increases file size if the whole library is loaded.
- Heavy overriding of default styles can become difficult; many nested classes make HTML verbose.

7. Frequently used Bootstrap classes (quick revision)

Category	Classes
Containers	container, container-fluid, container-md
Spacing	m-3, mt-2, mb-4, mx-auto, p-3, py-5, g-3 (gutter)
Text	text-center, text-primary, fw-bold, fs-1 ... fs-6, text-uppercase
Colours / background	bg-dark, bg-light, text-white, bg-success, text-danger
Buttons	btn btn-primary, btn-outline-danger, btn-lg, btn-sm
Flex utilities	d-flex, justify-content-between, align-items-center, flex-column
Display	d-none, d-md-block, d-inline
Images	img-fluid, img-thumbnail, rounded, rounded-circle
Components	navbar, card, alert, badge, modal, carousel, accordion, table

Q.3 (b)

Question: What do you mean by transform in CSS? Explain all the methods of 2d and 3d transform in detail. 3d transform an element into 60 degrees and move 200px from the original position where the origin of the transformation is center-center.

10 Marks | CO2

Answer:

1. Meaning of transform in CSS

The CSS3 **transform** property lets us **visually change** an element by moving, rotating, scaling or skewing it, without changing the layout of the neighbouring elements (the element still occupies its original space).

Transforms are of two kinds: **2D transforms** (in the X-Y plane) and **3D transforms** (which add the Z-axis, i.e. depth). Transforms are hardware-accelerated by the GPU so they are ideal for smooth animations.

Syntax: `transform: function(value) function(value) ...;` Several functions can be written in one declaration; they are applied in order.

2. 2D transform methods

Method	Description	Example
<code>translate(x, y)</code>	Moves the element along X and Y from its current position. Also <code>translateX()</code> and <code>translateY()</code> .	<code>transform: translate(50px, 100px);</code>
<code>rotate(angle)</code>	Rotates clockwise for a positive angle, anticlockwise for negative (units: deg, rad, turn).	<code>transform: rotate(20deg);</code>
<code>scale(x, y)</code>	Increases or decreases size. 1 = original, >1 enlarge, <1 shrink. Also <code>scaleX()</code> and <code>scaleY()</code> .	<code>transform: scale(2, 0.5);</code>
<code>skew(x-angle, y-angle)</code>	Tilts the element along X and Y. Also <code>skewX()</code> and <code>skewY()</code> .	<code>transform: skew(20deg, 10deg);</code>
<code>matrix(a, b, c, d, e, f)</code>	Combines all 2D methods in one: <code>scaleX</code> , <code>skewY</code> , <code>skewX</code> , <code>scaleY</code> , <code>translateX</code> , <code>translateY</code> .	<code>transform: matrix(1, 0, 0, 1, 30, 30);</code>

3. 3D transform methods

Method	Description	Example
<code>rotateX(angle)</code>	Rotates about the X-axis (horizontal axis) - element flips forward/backward.	<code>transform: rotateX(60deg);</code>
<code>rotateY(angle)</code>	Rotates about the Y-axis (vertical axis) - element turns left/right like a door.	<code>transform: rotateY(60deg);</code>
<code>rotateZ(angle)</code>	Rotates about the Z-axis (same as 2D <code>rotate()</code>).	<code>transform: rotateZ(30deg);</code>
<code>rotate3d(x, y, z, a)</code>	Rotates around a custom axis vector by angle a.	<code>transform: rotate3d(1, 1, 0, 45deg);</code>
<code>translate3d(x, y, z)</code>	Moves along X, Y and Z. Also <code>translateZ()</code> .	<code>transform: translate3d(10px, 20px, 30px);</code>
<code>scale3d(x, y, z)</code>	Scales in three dimensions. Also <code>scaleZ()</code> .	<code>transform: scale3d(1, 1.5, 2);</code>
<code>perspective(n)</code>	Gives depth: distance between user and Z=0 plane. Smaller value = stronger 3D effect.	<code>transform: perspective(500px);</code>
<code>matrix3d(...16 values)</code>	Combines all 3D transformations in a 4 x 4 matrix.	advanced use

4. Related transform properties

Property	Purpose
<code>transform-origin</code>	Sets the point about which the transformation happens. Default is <code>50% 50%</code> (center center). Values: keywords (left, top, center...), lengths, percentages.
<code>transform-style</code>	<i>flat</i> or <i>preserve-3d</i> - decides if children are positioned in 3D space.
<code>perspective</code>	Applied on the parent element to give all children a 3D depth effect.
<code>perspective-origin</code>	Position from which the viewer looks at the element (default <code>50% 50%</code>).
<code>backface-visibility</code>	<i>visible</i> or <i>hidden</i> - whether the back side of a rotated element is shown.
<code>transition</code>	Animates the change of the transform smoothly over a duration.

5. Program: rotate the element by 60 degrees and move it by 200px (origin center-center)

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>3D Transform</title>
<style>
  .scene {
    width: 700px; height: 300px; margin: 40px; /* parent gives the 3D depth */
    perspective: 600px;
    border: 1px dashed grey;
  }
  .box {
    width: 150px; height: 150px;
    background: lightblue; border: 2px solid navy;
    text-align: center; line-height: 150px; font-weight: bold;
  }
  .original { position: absolute; opacity: .4; } /* shows original position */

  .moved {
    transform-origin: center center; /* origin = center-center */
    transform: translateX(200px) rotateY(60deg); /* move 200px + rotate 60 */
    background: orange;
  }
</style>
</head>
<body>
  <div class="scene">
    <div class="box original">Original</div>
    <div class="box moved">3D Box</div>
  </div>
</body>
</html>
```

6. Explanation

- **transform-origin: center center** (equivalent to 50% 50%) makes the rotation happen about the middle of the box, so the box turns around its own centre.
- **rotateY(60deg)** rotates the element 60 degrees around the vertical Y-axis; the right edge moves away from the viewer, giving a 3D look. (Use *rotateX(60deg)* for a horizontal-axis tilt.)
- **translateX(200px)** moves the element 200px to the right from its original position.
- **perspective: 600px** on the parent .scene creates the depth needed to see the 3D effect; without it the rotation would look like a flat squeeze.
- **Order matters:** transforms are combined from right to left in terms of coordinate systems. Writing *translateX(200px) rotateY(60deg)* moves the box exactly 200px on the screen and then rotates it about its own centre. Writing *rotateY(60deg) translateX(200px)* would move the element along the already rotated X-axis, so it would not end up exactly 200px to the right.
- A shorthand using a single function is also possible: *transform: translate3d(200px,0,0) rotate3d(0,1,0,60deg);*

7. Example - transition with hover

```
.box { transition: transform 1s; }
.box:hover { transform: translateX(200px) rotateY(60deg); }
```

Q.4 (a)

Question: What are media queries in CSS? Create a responsive web page for paged-media query without using bootstrap.

10 Marks | CO2

Answer:

1. What are media queries?

A **media query** is a CSS3 technique that applies a block of CSS rules **only if** a given condition is true - for example when the viewport is narrower than 600px, when the device is in landscape orientation, or when the page is printed. It is the core of **Responsive Web Design (RWD)** because the same HTML can adapt itself to different screen sizes and output media without changing the content.

Syntax:

```
@media media-type and (media-feature) {  
    /* CSS rules that apply only when the condition is true */  
}  
/* logical operators: and , not , only , comma (or) */  
@media screen and (min-width: 600px) and (max-width: 900px) { ... }
```

2. Media types

Media type	Meaning
all	Default - all devices.
screen	Computer screens, tablets, smartphones.
print	Printers and print-preview (paged media).
speech	Screen readers that read the page aloud.

3. Common media features

Feature	Description
width, min-width, max-width	Width of the viewport. <i>min-width</i> = mobile-first approach; <i>max-width</i> = desktop-first approach.
height, min-height, max-height	Height of the viewport.
orientation	<i>portrait</i> or <i>landscape</i> .
resolution	Pixel density, e.g. <i>min-resolution: 2dppx</i> (Retina).
aspect-ratio	Ratio of width to height of the viewport.
prefers-color-scheme	<i>dark</i> or <i>light</i> mode of the operating system.
prefers-reduced-motion	User asked to reduce animations.
hover, pointer	Whether the device supports hover / a fine pointer (mouse) or coarse (touch).

4. Typical breakpoints

Device	Width	Media query
Phones	up to 600px	@media (max-width: 600px)
Tablets	601px - 992px	@media (min-width: 601px) and (max-width: 992px)
Laptops / desktops	993px and above	@media (min-width: 993px)

5. What is paged media?

Paged media means output that is split into discrete pages, such as a printed document, PDF or print preview - as opposed to continuous scrolling media like a screen. CSS handles it with **@media print** and the **@page** rule, which controls the page size, margins and page breaks. Common properties are:

Property / rule	Use
@page { size: A4; margin: 2cm; }	Sets paper size, orientation and margins of every printed page.
@page :first { margin-top: 4cm; }	Pseudo-classes :first, :left, :right select particular pages.
page-break-before / after	<i>always, avoid, auto</i> - forces or avoids a break before/after an element (modern name: break-before, break-after).
page-break-inside: avoid	Prevents an element (table row, figure) from being split across pages.
orphans / widows	Minimum lines left at the bottom / top of a page.

6. Program: responsive page with screen and print (paged media) queries - no Bootstrap

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive Page with Media Queries</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; font-family: Arial, sans-serif; }
  header, footer { background: #1f3a5f; color: #fff; padding: 15px; text-align: center; }
  nav { background: #2b6cb0; }
  nav a { color: #fff; text-decoration: none; display: block; padding: 10px; }
  .container { display: flex; flex-wrap: wrap; } /* default: desktop */
  .menu { width: 20%; background: #eaf1fb; padding: 15px; }
  .content { width: 60%; padding: 15px; }
  .ads { width: 20%; background: #fff3cd; padding: 15px; }
  nav a { display: inline-block; }

  /* ----- Tablet: 601px to 992px ----- */
  @media screen and (min-width: 601px) and (max-width: 992px) {
    .menu { width: 30%; }
    .content { width: 70%; }
    .ads { width: 100%; }
  }

  /* ----- Phone: up to 600px ----- */
  @media screen and (max-width: 600px) {
    .menu, .content, .ads { width: 100%; }
    nav a { display: block; border-bottom: 1px solid #fff; }
    header h1 { font-size: 20px; }
  }

  /* ----- Landscape orientation ----- */
  @media (orientation: landscape) and (max-height: 500px) {
    header { padding: 5px; }
  }

  /* ----- PAGED MEDIA : printing ----- */
  @page { size: A4; margin: 2cm; }
  @page :first { margin-top: 3cm; }
  @media print {
    nav, .menu, .ads, footer button { display: none; } /* hide non-essential parts */
    .content { width: 100%; font-size: 12pt; color: #000; }
    header { background: none; color: #000; border-bottom: 2px solid #000; }
    h2 { page-break-before: always; } /* every h2 on a new page */
    table, img, figure { page-break-inside: avoid; }
    a::after { content: " (" attr(href) ")"; font-size: 10pt; } /* show URLs */
  }
</style>
</head>
<body>
  <header><h1>My Responsive Website</h1></header>
  <nav>
    <a href="#home">Home</a><a href="#about">About</a>
    <a href="#services">Services</a><a href="#contact">Contact</a>
  </nav>
  <div class="container">
    <aside class="menu"><h3>Menu</h3><p>Links, categories...</p></aside>
    <main class="content">
      <h2>Welcome</h2>
      <p>Resize the browser window or press Ctrl+P (print preview) to see how the
        layout changes for tablet, phone and printed pages.</p>
      <h2>About Media Queries</h2>
      <p>Media queries apply CSS conditionally on screen size, orientation, resolution
        and output medium.</p>
    </main>
    <aside class="ads"><h3>Advertisement</h3><p>Sponsored content</p></aside>
  </div>
  <footer>&copy; 2022 MCA Department <button onclick="window.print()">Print</button></footer>
</body>
</html>
```

7. Explanation

- **viewport meta tag** tells mobile browsers to use the device width, otherwise media queries based on width do not work as expected.
- **Default (desktop) layout:** three columns (20% - 60% - 20%) built using Flexbox, no Bootstrap.
- **Tablet query (601-992px):** menu 30%, content 70%, advertisement moves to a full-width row below.
- **Phone query (max 600px):** all blocks become 100% wide so they stack one below another; the navigation links become vertical.
- **@media print + @page** is the *paged media* part: it sets A4 paper with 2cm margins, hides the navigation, sidebars and ads, keeps black text on white, starts each h2 heading on a new page, avoids splitting tables/images across two pages, and prints link URLs after link text.

Q.4 (b)

Question: What is CSS3 gradient? Explain its types in detail with suitable example.

10 Marks | CO1

Answer:

1. What is a CSS3 gradient?

A **CSS3 gradient** lets us display a smooth, progressive transition between two or more specified colours **without using image files**. Gradients are generated by the browser and are treated as an **image** value, so they are used with `background`, `background-image`, `border-image` or `list-style-image`, not with `background-color`. Advantages: faster page loading (no extra HTTP request), scalable to any size without pixelation, and easy to edit.

2. Types of CSS3 gradients

Type	Function	Description
Linear gradient	<code>linear-gradient()</code>	Colours transition along a straight line (top to bottom by default).
Radial gradient	<code>radial-gradient()</code>	Colours radiate outward from a centre point in a circle or ellipse.
Conic gradient	<code>conic-gradient()</code>	Colours rotate around a centre point, like a pie chart or colour wheel.
Repeating gradients	<code>repeating-linear-gradient()</code> <code>repeating-radial-gradient()</code> <code>repeating-conic-gradient()</code>	Repeat the colour stops to fill the element - used for stripes and patterns.

3. Linear gradient

Syntax: `background-image: linear-gradient(direction, color-stop1, color-stop2, ...);`

The direction can be given as keywords (*to right*, *to bottom*, *to bottom right*) or as an angle (*0deg* = to top, *90deg* = to right, *180deg* = to bottom). Colour stops may have positions in % or px.

```
/* 1. default: top to bottom */
#g1 { background: linear-gradient(red, yellow); }

/* 2. left to right */
#g2 { background: linear-gradient(to right, red, yellow); }

/* 3. diagonal */
#g3 { background: linear-gradient(to bottom right, red, yellow); }

/* 4. using an angle */
#g4 { background: linear-gradient(45deg, red, blue); }

/* 5. multiple colour stops with positions (rainbow) */
#g5 { background: linear-gradient(to right, red 0%, orange 20%, yellow 40%,
                                   green 60%, blue 80%, violet 100%); }

/* 6. transparency using rgba() */
#g6 { background: linear-gradient(to right, rgba(255,0,0,0), rgba(255,0,0,1)); }
```

4. Radial gradient

Syntax: `radial-gradient(shape size at position, color-stop1, color-stop2, ...);`

- **shape:** *ellipse* (default) or *circle*.
- **size:** *closest-side*, *farthest-side*, *closest-corner*, *farthest-corner* (default).

- **position:** keywords or values, e.g. *at center, at top left, at 30% 60%*.

```
#r1 { background: radial-gradient(red, yellow, green); }           /* evenly spaced */
#r2 { background: radial-gradient(red 5%, yellow 30%, green 70%); } /* stops set */
#r3 { background: radial-gradient(circle, white, blue); }         /* circle */
#r4 { background: radial-gradient(circle closest-side at 30% 40%, yellow, red); }
```

5. Conic gradient

Syntax: conic-gradient(from angle at position, color degree, ...); Colours are placed around a centre starting from the top (12 o'clock).

```
#c1 { background: conic-gradient(red, yellow, green, blue, red); } /* colour wheel */
#c2 { width: 200px; height: 200px; border-radius: 50%;           /* pie chart */
      background: conic-gradient(red 0deg 90deg, green 90deg 200deg, blue 200deg 360deg); }
#c3 { background: conic-gradient(from 45deg at 30% 60%, orange, purple); }
```

6. Repeating gradients

```
/* diagonal stripes */
#s1 { background: repeating-linear-gradient(45deg, red, red 10px, white 10px, white 20px); }

/* concentric rings */
#s2 { background: repeating-radial-gradient(circle, blue, blue 10px, white 10px, white 20px); }

/* repeating conic - sunburst */
#s3 { background: repeating-conic-gradient(red 0 15deg, yellow 15deg 30deg); }
```

7. Complete example (HTML)

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8"><title>CSS3 Gradients</title>
<style>
  div { width: 220px; height: 120px; margin: 10px; display: inline-block;
        color: #fff; text-align: center; line-height: 120px; font-weight: bold; }
  .lin { background: linear-gradient(to right, #ff416c, #ff4b2b); }
  .rad { background: radial-gradient(circle, yellow, green); }
  .con { background: conic-gradient(red, yellow, lime, aqua, blue, magenta, red); }
  .rep { background: repeating-linear-gradient(45deg, #333, #333 10px, #999 10px, #999 20px); }
</style>
</head>
<body>
  <div class="lin">Linear</div>
  <div class="rad">Radial</div>
  <div class="con">Conic</div>
  <div class="rep">Repeating</div>
</body>
</html>
```

8. Quick comparison

Point	Linear	Radial	Conic
Colour changes	Along a line	From the centre outward	Around the centre (angle)
Controlled by	Direction / angle	Shape, size, position	Start angle, position
Typical use	Buttons, headers, backgrounds	Spotlights, buttons, glows	Pie charts, colour wheels

Q.5 (a)

Question: Explain in detail the HTML5 audio and video elements with all the attributes. How to load the audio and video loop in your web page?

10 Marks | CO2

Answer:

1. Introduction

Before HTML5, playing audio or video in a page required third-party plug-ins such as Adobe Flash or QuickTime. HTML5 introduced the `<audio>` and `<video>` elements, which allow the browser to play media natively, with built-in controls and a JavaScript API. Both elements can contain **fallback text** that is shown by browsers which do not support them, and multiple `<source>` children so the browser can pick the first format it understands.

2. The audio element

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
  Your browser does not support the audio element.
</audio>
```

Supported audio formats

Format	MIME type	Notes
MP3	audio/mpeg	Most widely supported.
Ogg (Vorbis)	audio/ogg	Open format; supported by Chrome, Firefox, Edge.
WAV	audio/wav	Uncompressed, large size, high quality.
AAC / M4A	audio/aac	Good quality at small size; Safari, Chrome.

3. The video element

```
<video width="480" height="270" controls poster="thumbnail.jpg" preload="metadata">
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.webm" type="video/webm">
  <source src="movie.ogv" type="video/ogg">
  <track src="english.vtt" kind="subtitles" srclang="en" label="English" default>
  Your browser does not support the video tag.
</video>
```

Supported video formats

Format	MIME type	Codecs	Support
MP4	video/mp4	H.264 video + AAC audio	All major browsers
WebM	video/webm	VP8/VP9 video + Vorbis/Opus audio	Chrome, Firefox, Edge, Opera
Ogg	video/ogg	Theora video + Vorbis audio	Chrome, Firefox, Opera

4. Attributes of the audio and video elements

Attribute	Applies to	Description
src	Both	URL of the media file (alternatively use child <source> elements).
controls	Both	Shows the browser's default controls: play/pause, seek bar, volume, fullscreen (video).
autoplay	Both	Starts playing as soon as it can. Most browsers block autoplay with sound unless the media is muted.
loop	Both	Restarts the media automatically when it ends.
muted	Both	Media starts with the sound turned off.
preload	Both	Hint on loading: <i>none</i> (do not load), <i>metadata</i> (only duration and size), <i>auto</i> (load the entire file).
crossorigin	Both	CORS setting for files loaded from another domain (anonymous / use-credentials).
width, height	Video	Size of the video display area in pixels.
poster	Video	Image displayed before the video starts or while it is downloading.
playsinline	Video	Plays inside the page on iOS instead of forcing fullscreen.
controlslist	Both	Hides specific controls: nodownload, nofullscreen, noremoteplayback.
disablepictureinpicture	Video	Disables the Picture-in-Picture button.
id, class, style	Both	Global attributes used for styling and scripting.

5. Child elements

Element	Purpose	Attributes
<source>	Specifies multiple media files; the browser uses the first supported one.	src, type, media
<track>	Adds text tracks - subtitles, captions, descriptions - in WebVTT (.vtt) format.	src, kind, srclang, label, default

6. How to load audio and video in a loop

Method 1 - using the loop attribute (simplest):

```
<h3>Audio in loop</h3>
<audio controls loop>
  <source src="song.mp3" type="audio/mpeg">
</audio>

<h3>Video in loop (muted autoplay, like a background video)</h3>
<video width="480" controls autoplay muted loop>
  <source src="movie.mp4" type="video/mp4">
</video>
```

Method 2 - using JavaScript (loop property and the ended event):

```
<video id="vid" width="480" controls>
  <source src="movie.mp4" type="video/mp4">
</video>
<button onclick="toggleLoop()">Toggle Loop</button>

<script>
  var v = document.getElementById("vid");
```

```

v.loop = true; // same as the loop attribute

// alternative: restart manually when the media ends
v.addEventListener("ended", function () {
    v.currentTime = 0;
    v.play();
});

function toggleLoop() { v.loop = !v.loop; alert("loop = " + v.loop); }
</script>

```

7. JavaScript media API (for revision)

Members	Description
play(), pause(), load()	Methods to start, pause or reload the media.
currentTime	Gets or sets the current playback position in seconds.
duration	Total length of the media in seconds (read-only).
volume, muted	Volume from 0.0 to 1.0; mute on/off.
playbackRate	Speed of playback (1 = normal, 2 = double).
paused, ended	Boolean flags describing the state.
Events	loadedmetadata, canplay, play, pause, timeupdate, ended, volumechange, error.

8. Complete example with custom controls

```

<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>Audio and Video</title></head>
<body>
    <h2>Audio (loop)</h2>
    <audio id="aud" controls loop preload="auto">
        <source src="song.mp3" type="audio/mpeg">
        Your browser does not support audio.
    </audio>

    <h2>Video (loop)</h2>
    <video id="vid" width="420" controls loop poster="thumb.jpg">
        <source src="movie.mp4" type="video/mp4">
        <source src="movie.webm" type="video/webm">
        Your browser does not support video.
    </video>
    <p>
        <button onclick="document.getElementById('vid').play()">Play</button>
        <button onclick="document.getElementById('vid').pause()">Pause</button>
        <button onclick="document.getElementById('vid').currentTime = 0">Restart</button>
    </p>
</body>
</html>

```

Q.5 (b)

Question: What do you mean by responsive web page? Create a responsive photo gallery using bootstrap.

10 Marks | CO2

Answer:

1. What is a responsive web page?

A **responsive web page** is a page whose layout, images and text automatically **adapt to the size, resolution and orientation** of the device on which it is viewed - mobile, tablet, laptop or large desktop - using a single HTML code base. The term **Responsive Web Design (RWD)** was coined by Ethan Marcotte (2010). Users get an easy-to-read and easy-to-navigate page without pinching, zooming or horizontal scrolling.

2. Main ingredients of responsive design

Technique	Explanation
Viewport meta tag	<code><meta name="viewport" content="width=device-width, initial-scale=1"></code> makes the page width equal to the device width.
Fluid grid	Layout widths are in percentages or fr units instead of fixed pixels (Bootstrap grid, CSS Grid, Flexbox).
Flexible images / media	<code>max-width: 100%; height: auto;</code> (Bootstrap: <code>img-fluid</code>) so images never overflow their container.
Media queries	CSS rules applied at different breakpoints (see Q.4 a).
Relative units	<code>%</code> , <code>em</code> , <code>rem</code> , <code>vw</code> , <code>vh</code> instead of <code>px</code> .
Mobile-first approach	Design for the smallest screen first, then add rules with <code>min-width</code> queries.

3. Advantages of responsive design

- One website for all devices - lower development and maintenance cost than a separate mobile site.
- Better user experience and lower bounce rate; helps SEO (Google prefers mobile-friendly pages).
- A single URL makes sharing and analytics easier.
- Future-proof - adapts to new screen sizes automatically.

4. Responsive photo gallery using Bootstrap 5

The gallery uses the Bootstrap **grid**: 1 image per row on phones, 2 on tablets, 3 on laptops and 4 on large screens. Images use `img-fluid`; hovering enlarges the picture and clicking opens a **modal** with the large photo.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Responsive Photo Gallery</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
  <style>
    .gallery img { width: 100%; height: 220px; object-fit: cover; cursor: pointer;
                  transition: transform .3s; }
    .gallery img:hover { transform: scale(1.06); }
  </style>
</head>
<body>
```

```

<nav class="navbar navbar-dark bg-dark mb-4">
  <div class="container"><span class="navbar-brand">My Photo Gallery</span></div>
</nav>

<div class="container gallery">
  <h2 class="text-center mb-4">Gallery</h2>

  <!-- 1 column on phones, 2 on tablets, 3 on laptops, 4 on large screens -->
  <div class="row g-3">
    <div class="col-12 col-sm-6 col-lg-4 col-xl-3">
      
    </div>
    <div class="col-12 col-sm-6 col-lg-4 col-xl-3">
      
    </div>
    <div class="col-12 col-sm-6 col-lg-4 col-xl-3">
      
    </div>
    <div class="col-12 col-sm-6 col-lg-4 col-xl-3">
      
    </div>
    <div class="col-12 col-sm-6 col-lg-4 col-xl-3">
      
    </div>
    <div class="col-12 col-sm-6 col-lg-4 col-xl-3">
      
    </div>
  </div>

  <!-- Modal (pop-up) showing the large picture -->
  <div class="modal fade" id="photoModal" tabindex="-1">
    <div class="modal-dialog modal-lg modal-dialog-centered">
      <div class="modal-content">
        <div class="modal-body p-0">
          <img id="bigImg" src="" class="img-fluid w-100" alt="Large view">
        </div>
      </div>
    </div>
  </div>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>
<script>
  function showImg(el) { document.getElementById("bigImg").src = el.src; }
</script>
</body>
</html>

```

5. Explanation

- **Viewport meta tag + Bootstrap CSS link** make the page mobile-first and load the framework.
- **container > row > col-*** is the grid. *col-12* = full width on phones, *col-sm-6* = 2 per row ($\geq 576\text{px}$), *col-lg-4* = 3 per row ($\geq 992\text{px}$), *col-xl-3* = 4 per row ($\geq 1200\text{px}$). *g-3* adds gutters (gaps) between photos.
- **img-fluid** makes each image scale within its column; *rounded* and *shadow-sm* add a rounded border and light shadow; *object-fit: cover* keeps all thumbnails the same size.
- **Modal:** *data-bs-toggle="modal"* and *data-bs-target* open the pop-up; the small JavaScript function copies the clicked image's source into the modal.
- The CSS *transform: scale(1.06)* with *transition* gives a zoom effect on hover.

6. Alternative: gallery with Bootstrap cards

```
<div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4">
  <div class="col">
    <div class="card h-100">
      
      <div class="card-body"><h5 class="card-title">Sunset</h5>
        <p class="card-text">Evening view of the hills.</p></div>
    </div>
  </div>
  <!-- repeat the .col block for more photos -->
</div>
```

Bonus Revision Notes - Full Syllabus Quick Review

These extra notes cover the related topics of the Full Stack Web Development syllabus (HTML5, CSS3, SVG/Canvas, Bootstrap) that examiners often ask as alternative questions.

A. New features of HTML5

Feature	Details
Semantic elements	<code><header></code> <code><nav></code> <code><section></code> <code><article></code> <code><aside></code> <code><footer></code> <code><main></code> <code><figure></code> <code><figcaption></code>
Multimedia	<code><audio></code> <code><video></code> <code><source></code> <code><track></code> - no plug-ins.
Graphics	<code><canvas></code> , inline SVG, MathML.
New form inputs	email, url, number, range, date, time, color, search, tel; attributes: placeholder, required, autofocus, pattern, min, max, list (datalist).
Web Storage	<i>localStorage</i> (permanent) and <i>sessionStorage</i> (until the tab closes); about 5 MB, more than cookies.
Geolocation API	<code>navigator.geolocation.getCurrentPosition()</code> to obtain latitude and longitude.
Drag and Drop	Native API with draggable attribute and <code>dataTransfer</code> (Q.1 a).
Web Workers	Run JavaScript in a background thread without freezing the UI.
Server-Sent Events / WebSocket	One-way and two-way real-time communication with the server.
Offline / cache	Application cache (obsolete) replaced by Service Workers and Cache API.
Other	<i>contenteditable</i> , <i>data-*</i> attributes, <code><details></code> , <code><progress></code> , <code><meter></code> , simplified doctype <code><!DOCTYPE html></code> .

B. New features of CSS3

Category	Properties
Selectors	<code>[attr^=x]</code> , <code>[attr\$=x]</code> , <code>[attr*=x]</code> , <code>:nth-child()</code> , <code>:not()</code> , <code>:first-of-type</code> , <code>::before</code> , <code>::after</code>
Box / borders	<code>border-radius</code> , <code>box-shadow</code> , <code>border-image</code> , <code>box-sizing</code>
Backgrounds	<code>background-size</code> , <code>background-origin</code> , <code>background-clip</code> , multiple backgrounds, gradients
Text	<code>text-shadow</code> , <code>text-overflow</code> , <code>word-wrap</code> , <code>word-break</code> , <code>@font-face</code> (web fonts)
Colours	<code>rgba()</code> , <code>hsl()</code> , <code>hsla()</code> , <i>opacity</i> .
Transforms and animation	<code>transform</code> , <code>transition</code> , <code>@keyframes</code> , <code>animation</code>
Layout	Flexbox (<code>display:flex</code>), CSS Grid (<code>display:grid</code>), multi-column layout
Responsive	<code>@media</code> queries, viewport units <code>vw/vh</code> , <code>min()/max()/clamp()</code>
Variables	<code>:root { --main: #333; } color: var(--main);</code>

C. CSS transitions and animations

```
/* transition: property duration timing-function delay */
button { background: blue; transition: background 0.5s ease-in 0s; }
button:hover { background: red; }

/* keyframe animation */
@keyframes slide { from { left: 0; } 50% { left: 150px; } to { left: 0; } }
.ball { position: relative; animation: slide 3s infinite linear; }
```

D. Flexbox vs CSS Grid

Flexbox	CSS Grid
One-dimensional layout (row OR column).	Two-dimensional layout (rows AND columns together).
Content-first: item sizes drive the layout.	Layout-first: the grid is defined, then items are placed.
Key properties: flex-direction, justify-content, align-items, flex-wrap, flex-grow.	Key properties: grid-template-columns/rows, gap, grid-area, place-items.
Best for navbars, toolbars, card rows, centring items.	Best for page layouts, galleries, dashboards.

E. Canvas basics

```
var ctx = document.getElementById("c").getContext("2d");
ctx.fillStyle = "green"; ctx.fillRect(10, 10, 100, 50); // filled rectangle
ctx.strokeStyle = "red"; ctx.strokeRect(10, 80, 100, 50); // outlined rectangle
ctx.beginPath(); ctx.moveTo(150, 10); ctx.lineTo(250, 100); ctx.stroke(); // line
ctx.beginPath(); ctx.arc(200, 100, 40, 0, 2 * Math.PI); ctx.fill(); // circle
ctx.font = "20px Arial"; ctx.fillText("Hello Canvas", 10, 160); // text
ctx.drawImage(img, 0, 0); // image
```

F. Bootstrap components (one-line notes)

Component	Key classes / attributes
Navbar	navbar navbar-expand-lg navbar-dark bg-dark; navbar-toggler; collapse navbar-collapse
Card	card, card-img-top, card-body, card-title, card-text
Carousel	carousel slide, carousel-inner, carousel-item active, data-bs-ride="carousel"
Modal	modal fade, modal-dialog, data-bs-toggle="modal", data-bs-target="#id"
Alerts	alert alert-success warning danger info, alert-dismissible
Forms	form-control, form-label, form-select, form-check, input-group, was-validated
Tables	table table-striped table-hover table-bordered table-responsive
Accordion / collapse	accordion, accordion-item, collapse, data-bs-toggle="collapse"

G. Most probable exam questions (practice list)

- Explain HTML5 semantic elements with an example page layout.
- Explain HTML5 form input types and new attributes with a registration form.
- Differentiate between HTML and HTML5; between CSS and CSS3.
- Explain HTML5 Web Storage (localStorage vs sessionStorage vs cookies) with an example.
- Explain the Geolocation API with a program to display latitude and longitude.
- What is Canvas? Draw a line, circle, and text using the canvas API.

- Explain CSS3 transitions and animations with keyframes.
- Explain Flexbox with all its properties; compare with CSS Grid.
- Create a responsive navbar or a login form using Bootstrap.
- Explain the Bootstrap carousel, modal and card components with code.